



New CEO for Uber

Former head of General Electric, Jeff Inmett could be the new CEO for Uber. Board voting expected to happen soon. <http://dgit.in/UberCE01>



New super slippery surface

A team of researchers have designed a new kind of slippery material that even mussels can't seem to get a grip on it. <http://dgit.in/musselslip>

tor class can have some any number of methods. Each method that is directly attached to the generator prototype is considered to be a task. Each task is run in sequence by the Yeoman environment run loop. In other words, every method that is in the class will get called.

This can be a problem in case you want to create some helper or private methods that you don't want called when other tasks are being executed. To create a private method you need to add `_` (an underscore) before the method name so that the Generator does not execute that helper method. Another way to create helper methods to add them to the instance variable – create a set of instance methods whenever you want to create some helper methods.

The Yeoman run loop prioritises methods using the Grouped-queue module. Priorities are defined in your code as special prototype method names. When a method name is the same as a priority name, the run loop pushes the method into this special queue. If the method name doesn't match a priority, it is pushed in the default group. The generator follows an available list of priority methods (in running order):

1. **initializing:** Your initialization methods (checking current project state, getting configs, etc)

2. **prompting:** Where you prompt users for options – you'd call `this.prompt()`
3. **configuring:** Saving configurations and configure the project (creating .editorconfig files and other meta-data files)
4. **default:** If the method name doesn't match a priority, it will be pushed to this group.
5. **Writing:** Where you write the generator specific files (routes, controllers, etc)
6. **conflicts:** Where conflicts are handled (used internally)
7. **install:** Where installations are run (`npm`, `bower`)
8. **end:** Called last, cleanup, say good bye, etc

Now these methods are called in sequence and if you want to add some method in some group you may group them likewise.

ASYNC

There are many ways to stop the yeoman run-loop and the easiest one is to return a promise. The run-loop will halt till the promise is resolved or will exit whenever the promise is rejected. Now if you are using some old libraries that do not support promise yet then you have to use `this.async` to halt the run-loop, like mentioned here: <http://dgit.in/AsyncCde>

HANDLING USER RESPONSE

Your generator will interact a lot with the end user and though Yeoman is primarily based on CLI, its sheer flexibility allows any generator to be run in any environment. Yeoman provides a set of abstractions to the end user, and it's your responsibility as an author to use the abstractions.

Yeoman uses the prompt module provided by `inquirer.js`. You should refer to their list of APIs for a list of options available to the user. The prompt method is asynchronous and returns a promise. You will need to return the promise from your task, and wait for its completion before running another.

In our example, you will first see the line for logging where `this.log` is used, and whenever logging some output you should also use `this.log` instead of `console.log` because that is in the current running context of the generator. In our example, we're asking the user about:

- Name
- Email id (with validation)
- The app's name
- The port value in which the user would like to create the application (default port: 3000)
- The license

Thus, after adding each of the

*YEO-SEFUL RESOURCES



Getting started with Yeoman

Apart from this article, get started with Yeoman with this helpful official tutorial that takes you through the basics of this scaffolding tool.

<http://dgit.in/LrnYeoman>



Building Apps With the Yeoman Workflow

This tutorial from EnvatoTuts introduces you to the awesome guy that Yeoman is, and how you should talk to him to build apps.

<http://dgit.in/YeoManTut>



An alternative scaffolding tool

Check out Slush.js, an alternative scaffolding tool that promises to get you up and running with your new project in a matter of seconds.

<http://dgit.in/SlushJS>